SQL Injection Test: Vulnerable vs Secure Search

Izzaldeen · 2026-10-20 · ASP.NET Core + EF Core · local SQL Server Express · 5 fictional products · same inputs sent to both endpoints

1. Not Secure: `GET /api/OliveOilProducts/search-vulnerable`

Implementation (input concatenated into the SQL string)

```
var sql = "SELECT ... FROM OliveOilProducts WHERE Name LIKE '%" + term + "%'";
db.OliveOilProducts.FromSqlRaw(sql);
```

Executed SQL (from EF Core log), input `' OR 1=1 --`

```
Parameters=[]
SELECT Id, Name, Region, PricePerLiter, QuantityInStock FROM OliveOilProducts
WHERE Name LIKE '%' OR 1=1 --%'
```

Input	Result	Why
`Nabali`	2 products	Normal search works.
`' OR 1=1 --`	**5 products (all)**	`'` closed the string, `OR 1=1` is always true, `--` commented out the rest. The input became SQL code.

2. Secure: `GET /api/OliveOilProducts/search-secure`

Implementation (EF Core parameterized raw SQL)

```
var pattern = "%" + term + "%";
db.OliveOilProducts.FromSqlInterpolated($"SELECT ... FROM OliveOilProducts WHERE Name LIKE
{pattern}");
```

Executed SQL (from EF Core log), input `' OR 1=1 --`

```
Parameters=[p0='%' OR 1=1 --%']
SELECT Id, Name, Region, PricePerLiter, QuantityInStock FROM OliveOilProducts
WHERE Name LIKE @p0
```

Input	Result	Why
`Nabali`	2 products	Same as the vulnerable method.
`' OR 1=1 --`	**0 products**	The SQL text stayed `LIKE @p0`. The input was sent separately as a value, so it was searched as plain text and matched nothing.

> **Conclusion:** with normal input both methods return the same result. With the crafted input, the vulnerable method returned the whole table because the input was concatenated into the SQL and changed its logic. The secure method was not affected because the query and the value are sent separately, so the input is always treated as data, never as code.